

" OrbitMind AI– Intelligent Productivity & Study Companion"

by

Md. Imroz Newaz

CSE 11220321052

Zeniya Naznin

CSE 11230121138

Fatema Parvin Kanta

CSE 11230121192

Md. Mahfuzur Rahman Hridoy

CSE 11230121186

WEEK 04 ASSIGNMENT

Course Title: Software Development III: AI-Based Software Project Development

Course Code: CSE 4104

Section: 7D | Spring 2026



Submitted to

Md. Riaz Mahmud

Assistant Professor

Department of Computer Science and Engineering

Northern University of Business and Technology, Khulna

Khulna 9100, Bangladesh

CONTENTS

Title Page	1
Contents	2
CHAPTER I – Team Information	3
1.1 Team Information Sheet	3
1.1.1 Team Members	3
CHAPTER II – Core System Architecture and API Design	3-6
2.1 Project Information	3-4
2.2 API Design Document	4-5
2.2.1 Authentication APIs	4
2.2.2 Task Management APIs	4
2.2.3 AI Integration APIs	5
2.3 AI Integration Workflow	5
2.3.1 AI Service/Model Selection	6
2.3.2 Input to the AI System	6
2.3.3 Output From the AI System	6
2.3.4 How AI Improves the Project	6
CHAPTER III – System Diagrams	7-11
3.1 System Architecture Diagram	7
3.2 ER Diagram	7
3.3 Updated Use Case Diagram	8
3.4 Activity Diagram	9
3.5 AI Workflow Flowchart	10
References	11

CHAPTER I

1.1 Team Information Sheet

Field	Details
Team Name	CSE4104-7D-T04
Section	7D
Project Title	OrbitMind AI — Smart Productivity & Study Assistant
Team Leader	Md. Imroz Newaz (ID: 11220321052)

1.1.1 Team Members

Member Name	Student ID	Role
Md. Imroz Newaz	11220321052	Team Leader & Database Manager
Zeniya Naznin	11230121138	Frontend Developer
Fatema Parvin Kanta	11230121192	Backend Developer
Md. Mahfuzur Rahman Hridoy	11230121186	AI Integration Lead

CHAPTER II

2.1 Project Information

Overview: OrbitMind AI is a comprehensive, full-stack productivity web application engineered specifically to address the academic pressures faced by modern university students. At its core, the system merges traditional task management capabilities with advanced Artificial Intelligence (powered by Google Gemini) to create a proactive study environment.

Rather than just acting as a passive to-do list, OrbitMind AI dynamically automates study planning, summarizes dense and lengthy lecture materials, and provides a conversational academic coach available 24/7. Built upon a decoupled Node.js and Express backend, communicating with a NoSQL MongoDB database, the platform ensures that student data is handled securely, efficiently, and at scale. Our primary goal is to leverage this architecture to reduce academic burnout and streamline the learning process.

2.2 API Design Document

To facilitate smooth and secure communication between our frontend client and the backend server, we have designed a robust set of RESTful API endpoints. Below is an outline of our core planned endpoints.

2.2.1 Authentication APIs

- **POST /api/auth/register**
 - Purpose: Handles the creation of new student accounts. It securely hashes the user's password using bcrypt before storing the record in the database.
 - Input Data (JSON Payload): full_name, email, password, university, department
 - Expected Output: A 201 Created status, a success message, the generated User ID, and a secure JWT session token for immediate login.

- **POST /api/auth/login**
 - Purpose: Authenticates returning students by verifying their credentials against the hashed database records.
 - Input Data (JSON Payload): email, password
 - Expected Output: A 200 OK status and a JWT session token to be stored in the client's local storage.

2.2.2 Task Management APIs

- **GET /api/tasks**

- Purpose: Retrieves a personalized list of all study tasks and assignments associated with the currently authenticated user.
- Input Data: HTTP Authorization Header containing the Bearer JWT Token.
- Expected Output: A JSON array containing the user's task objects (including titles, deadlines, priorities, and completion statuses).
- **POST /api/tasks**
 - Purpose: Allows students to create and save a new academic task to their dashboard.
 - Input Data (JSON Payload): title, description, deadline, priority_level, estimated_hours
 - Expected Output: The newly inserted task object and a 201 Created status code.

2.2.3 AI Integration APIs

- **POST /api/ai/study-plan**
 - Purpose: Takes raw syllabus data and exam dates from the student, proxies it to the Gemini API, and returns a structured, day-by-day study roadmap.
 - Input Data (JSON Payload): subject_name, topics_array, target_exam_date, hours_per_day
 - Expected Output: A parsed JSON object containing the daily study schedule and a 200 OK status.
- **POST /api/ai/summarize**
 - Purpose: Condenses long, complex lecture transcripts into easily readable bullet points and short summaries.
 - Input Data (JSON Payload): original_content (A long string of text)
 - Expected Output: A JSON object containing generated_summary (a brief paragraph) and key_bullet_points (an array of core concepts).

2.3 AI Integration Workflow

Why AI is Necessary: Traditional productivity applications are inherently passive; they rely entirely on the user to manually calculate how many hours they should study per day or how to break down complex topics. In contrast, university coursework is dynamic and often

overwhelming. AI is strictly necessary in OrbitMind AI because it introduces a layer of active intelligence. By processing natural language, the system can act as a virtual tutor—intelligently distributing study workloads over time, simplifying dense academic texts, and providing context-aware advice. This directly addresses our project's core objective: reducing student burnout.

2.3.1 AI Service/Model Selection

We have integrated the Google Gemini API (via the official [@google/generative-ai](#) SDK). This model was selected due to its high accuracy in natural language comprehension, rapid processing speeds, and excellent ability to format responses into structured JSON data, which our backend requires.

2.3.2 Input to the AI System

The system feeds the AI various forms of data depending on the module. This includes user-provided conversational text for the Chatbot, array-based syllabus topics and deadline constraints for the Study Planner, or large blocks of raw textual transcripts for the Note Summarizer.

2.3.3 Output from the AI System

The Gemini API returns contextually generated text strings for conversations, deeply structured JSON arrays for daily study roadmaps, and concise paragraph summaries for note compression.

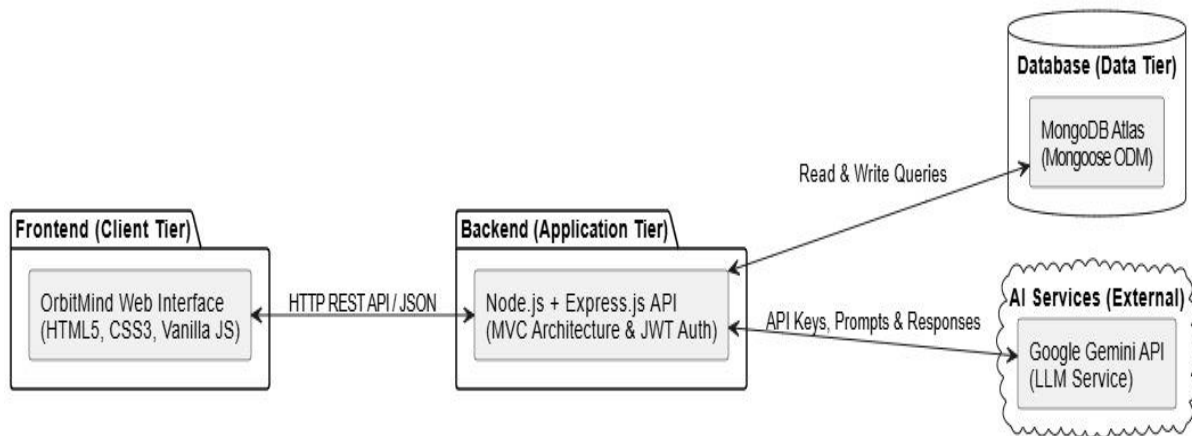
2.3.4 How AI Improves the Project

The integration of AI completely transforms OrbitMind from a standard CRUD (Create, Read, Update, Delete) application into a smart academic assistant. It automates the most time-consuming parts of studying—planning and reading—saving students countless hours and significantly improving their learning efficiency.

CHAPTER III

3.1 System Architecture Diagram

Theoretical Description: The System Architecture Diagram illustrates the high-level structural blueprint of the OrbitMind AI platform. We have adopted a modern, decoupled three-tier architecture. The Frontend Client Tier (built with responsive HTML5, CSS3, and Vanilla JS) manages the user interface and captures user inputs. The Backend Application Tier runs on a Node.js server using Express.js, handling all REST API routing, MVC business logic, and secure JWT authentication. The Data Tier utilizes MongoDB Atlas to ensure persistent, schema-less storage. Additionally, the diagram highlights our external cloud integration, demonstrating how our secure backend acts as a proxy to send engineered prompts to the Google Gemini API, returning intelligent responses back down the chain to the user interface.

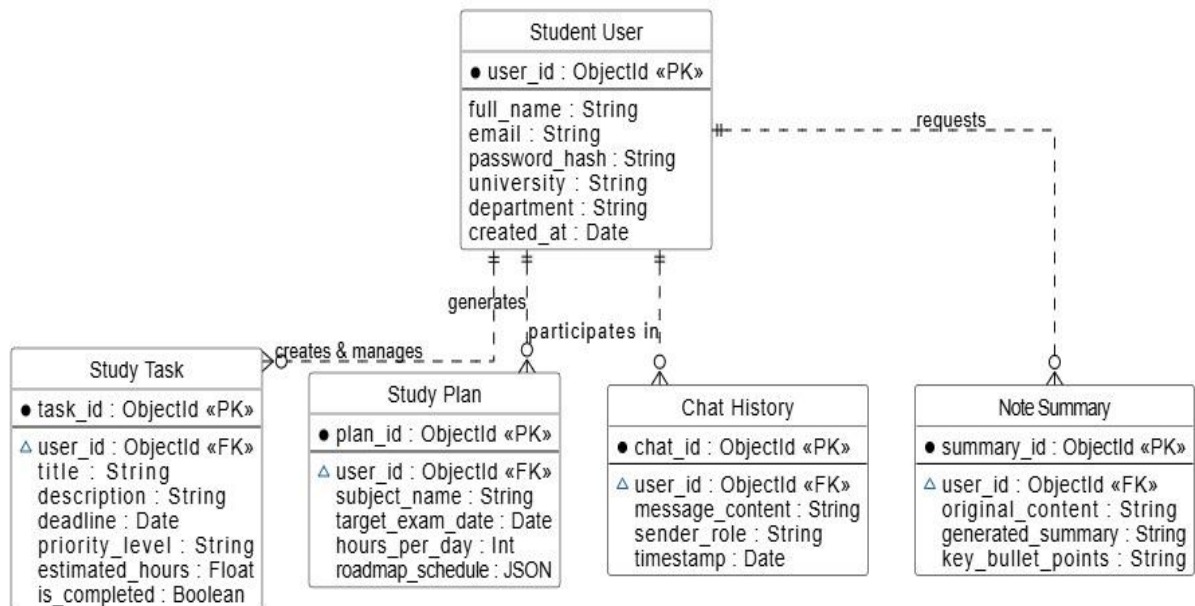


3.1.1 System Architecture Diagram

3.2 ER Diagram (Database Design)

Theoretical Description: The Entity Relationship (ER) Diagram models the logical database schema designed for our MongoDB environment. It defines the core data entities that drive the application: Student User, Study Task, Study Plan, Chat History, and Note Summary. To maintain relational integrity within a NoSQL structure, we utilize Primary Keys (<<PK>>) formatted as ObjectIDs to uniquely identify every document. Foreign Keys (<<FK>>) are established to map the one-to-many cardinality relationships—visually demonstrating that a

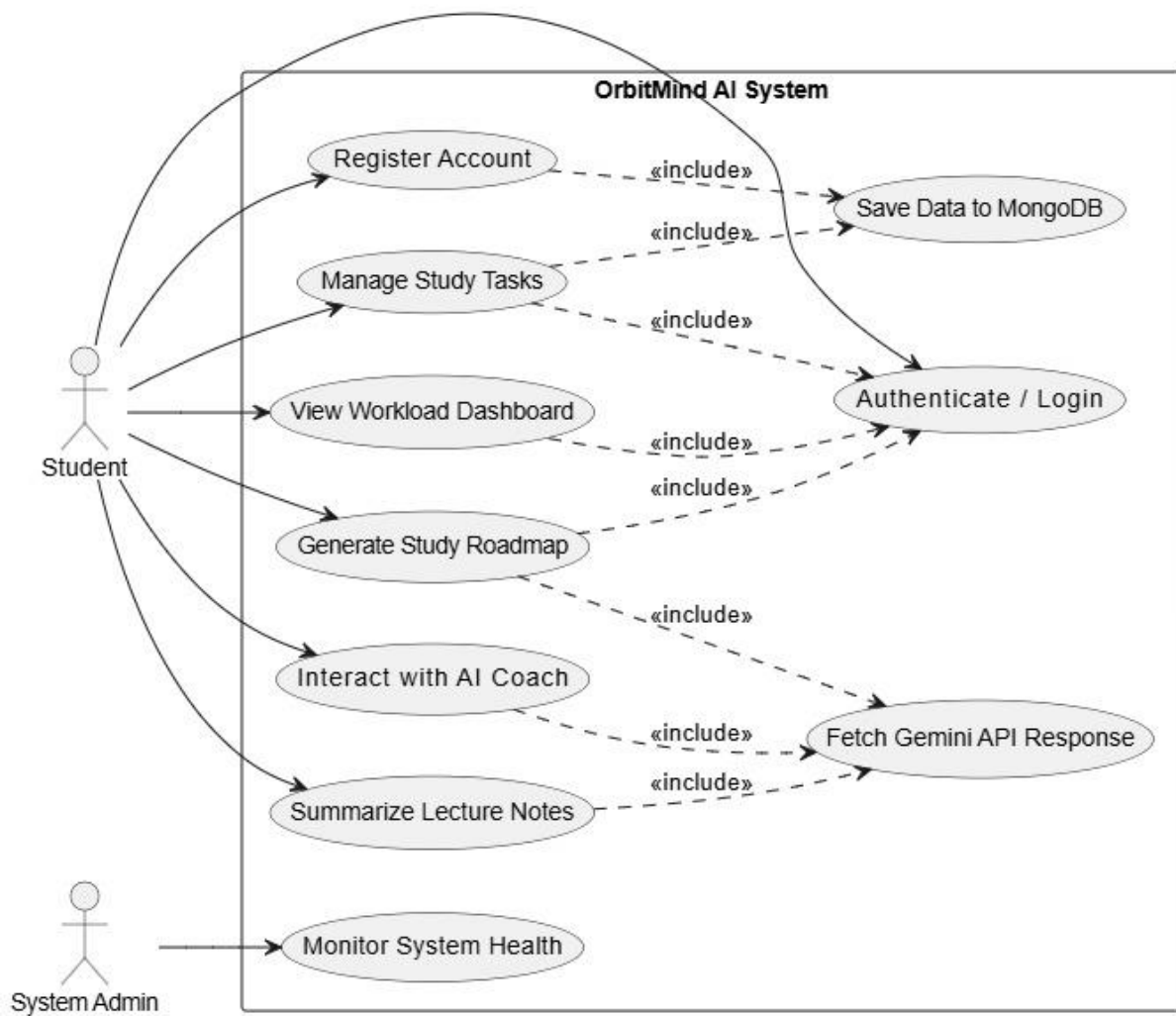
single authenticated student is the owner of multiple tasks, custom study plans, and historical chat logs. This structure guarantees strict data isolation, ensuring students only have access to their own academic data.



3.2.1 ER Diagram (Database Design)

3.3 Updated Use Case Diagram

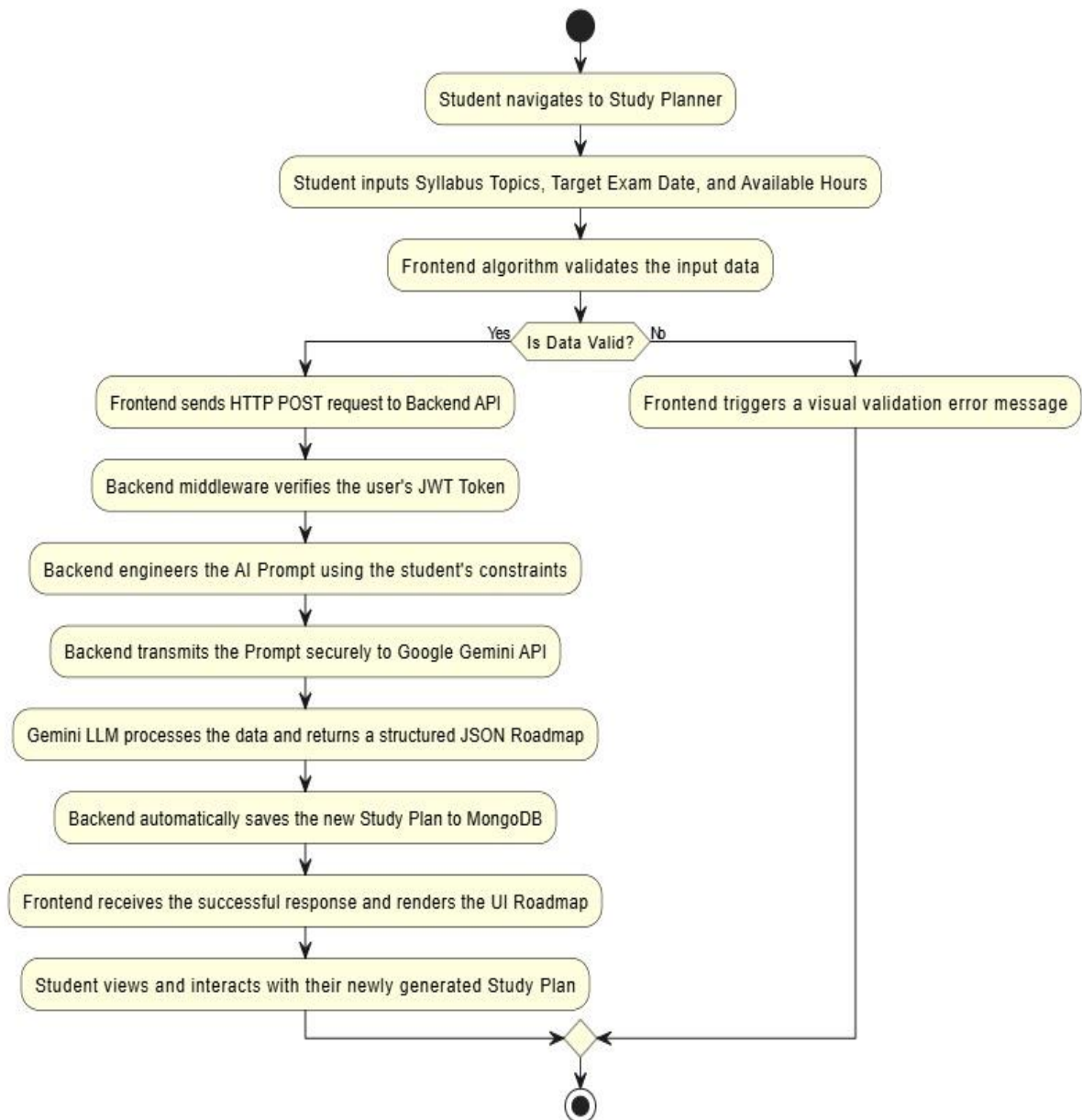
Theoretical Description: Based on architectural feedback from Week 3, the Use Case Diagram has been significantly updated to reflect a deeper level of system interaction. It maps the functional boundaries of the OrbitMind AI system by illustrating how the primary actors (Students and System Administrators) trigger various system processes. We have heavily utilized <<include>> dependencies to demonstrate mandatory backend logic. For example, the diagram now explicitly shows that "Authenticating / Login" is an absolute prerequisite for accessing the dashboard or managing tasks. Furthermore, any interaction with the AI Coach or the Study Planner now visually triggers backend sub-processes, such as "Fetching the Gemini API Response" and "Saving Data to MongoDB," accurately reflecting the true technical flow of the software.



3.3.1 Updated Use Case Diagram

3.4 Activity Diagram (AI Study Plan Workflow)

Theoretical Description: The Activity Diagram provides a sequential, behavioral view of one of our system's most complex major features: The AI Study Plan Generation Workflow. It traces the exact execution path a student takes, beginning with navigating to the planner module and submitting their syllabus constraints. The diagram carefully highlights crucial decision points, specifically the validation of user input. If the data passes frontend validation, the diagram illustrates the complex background processing that occurs—verifying the JWT token, engineering the API prompt, waiting for the Gemini model to process the data, saving the generated schedule to the database, and finally rendering the visual roadmap on the student's dashboard.

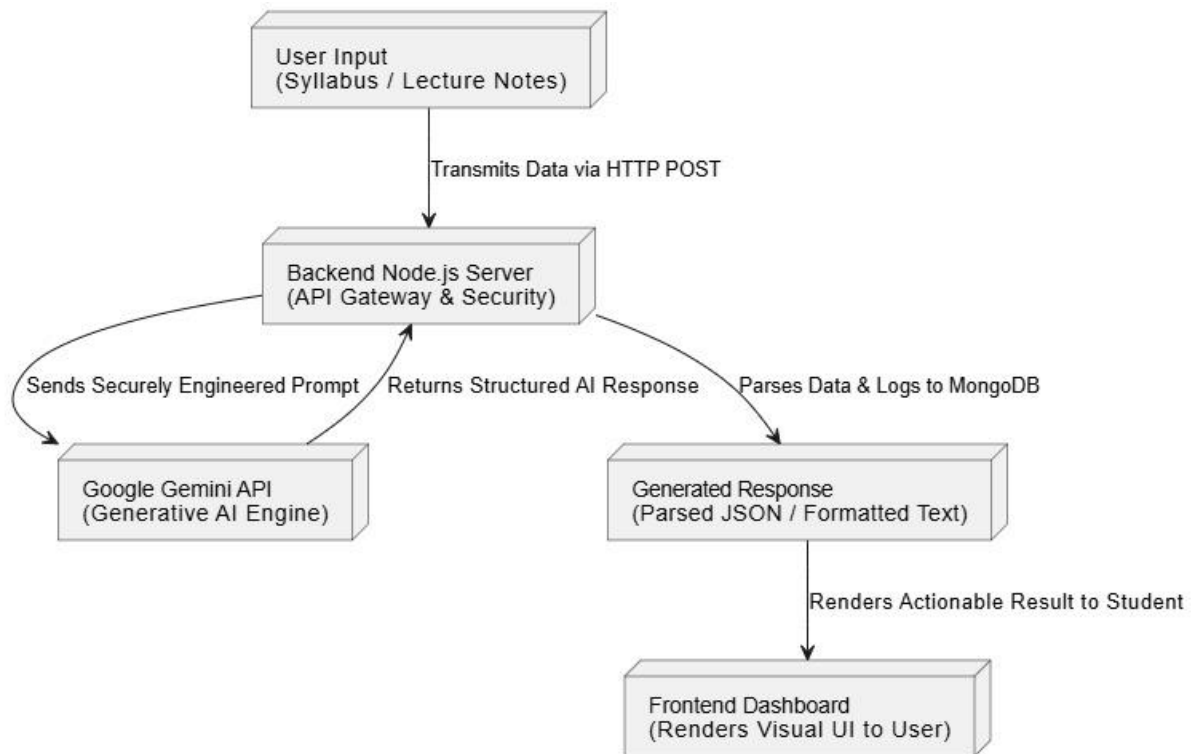


3.4.1 Activity Diagram (AI Study Plan Workflow)

3.5 AI Workflow Flowchart

Theoretical Description: To fulfill the mandatory AI Integration requirements, this flowchart visualizes the isolated pipeline of our Artificial Intelligence module. It breaks down the network topography of an AI request, showing how raw, unstructured user input is collected from the client interface and securely transmitted to the Node.js backend. Rather than exposing our API keys on the client side, the backend acts as a secure intermediary. It reformats the input

into a highly specific engineered prompt, communicates with the Google Gemini API, receives the output, parses it for safety and structure, stores a copy in the database for historical logging, and finally serves the polished result back to the frontend display.



3.5.1 AI Workflow Flowchart

References

Google Gemini API Official Documentation: <https://ai.google.dev/docs>

Express.js REST API Architecture Guidelines: <https://expressjs.com/>

Mongoose & MongoDB Logical Schema Design: <https://mongoosejs.com/>

Course Handbook: CSE 4104 Software Development III (Md. Riaz Mahmud)

CHAPTER III: Required System Diagrams: <https://app.diagrams.net/>